

# Java Fundamentals Made Visual

Tokens • Variables • Data Types • Operators •  
Casting • Input/Output

A visual beginner guide built from your classroom notes, with corrected Java examples, diagrams, code walkthroughs and lightweight practice.

Learn

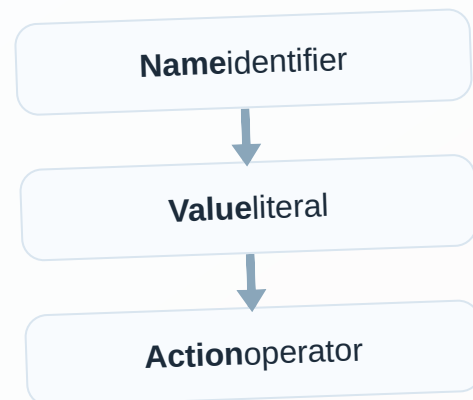
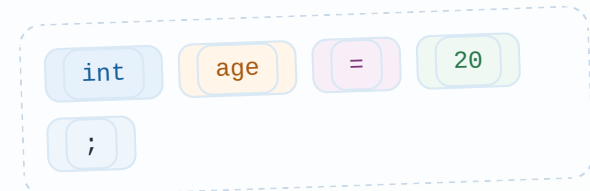
Visualize

Code

Practice

## ONE JAVA STATEMENT

```
int age = 20;
```



# What you will learn

This module explains the small building blocks that make Java code readable, meaningful and executable.

## LANGUAGE PIECES

### Tokens, keywords & identifiers

Break a statement into the units Java recognizes, then learn how programmers name things consistently.

## STORED DATA

### Variables, data types & literals

Understand how Java stores values and how the selected type controls what can be stored.

## OPERATIONS

### Operators & expressions

Perform calculations, comparisons, updates, logical checks and bit-level operations.

## CONVERSION & INTERACTION

### Type casting, Scanner & output

Convert compatible numeric types, read values from the keyboard and display results clearly.

## Recommended study order

1. Read the idea understand the definition in simple language



2. Inspect the visual see how the parts connect



3. Type the code do not only copy and paste



4. Predict before running train your reasoning

## ACCURACY NOTE

A few classroom-note examples contained small Java mistakes. They are corrected in this guide so the learning material remains technically accurate.

# Tokens

A **token** is the smallest individual unit of a Java program that the compiler can recognize as a meaningful language element.

```
int age = 20;
```

`int``age``=``20``;`

## 1 - KEYWORD

**int** is a reserved word with a predefined meaning.

## 2 - IDENTIFIER

**age** is a programmer-created name.

## 3 - LITERAL

**20** is a fixed value written directly in the program.

## 4 - OPERATOR + SEPARATOR

**=** performs assignment; **;** separates/terminates the statement.

## Five token categories

Keywords

Identifiers

Literals

Operators

Separators

### SEPARATORS YOU WILL OFTEN SEE

`( ) [ ] { } . ; ,`

Example: `System.out.println("Hello");` contains dots, parentheses and a semicolon.

**Quick Check:** Break `int mark = 30;` into its five tokens.

# Keywords

Java keywords are reserved words with predefined meaning. You cannot reuse them as your own class, method or variable names.

**DATA TYPES**

byte, short, int, long, float, double, char, boolean

**ACCESS**

public, private, protected

**CLASSES & INTERFACES**

class, interface, extends, implements

**OBJECTS**

new, this, super

**DECISION MAKING**

if, else, switch, case, default

**LOOPS & FLOW**

for, while, do, break, continue

**EXCEPTION HANDLING**

try, catch, finally, throw, throws

**RETURN**

return

**COMMON MISTAKE**

```
int class = 5;    // invalid: class is a keyword
```

**REMEMBER**

You do not need to memorize every Java keyword immediately. Learn what a keyword means when it appears in a topic and recognize that reserved words cannot be identifiers.

# Identifiers & naming conventions

An **identifier** is a name created by the programmer. Naming conventions are shared style rules that make code easier to read, maintain and review as a team.

## Identifier rules

- Can contain letters, digits, \_ and \$.
- Cannot begin with a digit.
- Cannot be a Java keyword.
- Java identifiers are case-sensitive.

### VALID

**student, student1, \$cost, \_marks**

### INVALID

**1student, @gmail, hash#1**

## Java naming conventions

| Program element | Convention       | Example        |
|-----------------|------------------|----------------|
| Class           | PascalCase       | StudentDetails |
| Interface       | PascalCase       | PaymentService |
| Method          | camelCase        | calculateTotal |
| Variable        | camelCase        | studentName    |
| Constant        | UPPER_SNAKE_CASE | MAX_MARKS      |
| Package         | lowercase        | com.godix.java |
| Object variable | camelCase        | studentOne     |

### WHY CONVENTIONS MATTER

If four developers write **StudentDetails**, **studentdetails**, **Studentdetails** and **studentDetails** for the same kind of concept, the codebase becomes harder to scan. Conventions create consistency.

# Variables

A variable is a named storage location used by a program to hold a value that can be read or changed during execution.

## ANALOGY

### A labeled bottle or container

**Variable** = the storage container.

**Identifier** = the name written on the container.

**Value** = what the container currently holds.

```
student1Mark -> 30  
student2Mark -> 30  
student3Mark -> 50
```

## 1 Declaration

```
int age;
```

Tell Java the type and variable name.

## 2 Initialization

```
age = 10;
```

Give the variable its first value.

## 3 Declaration + initialization

```
int age = 20;
```

## 4 Updating

```
age = 21;
```

Replace the previous value with a new one.

## Basic variable categories

### LOCAL

Declared inside a method, constructor or block. Used only within its scope.

### INSTANCE

Belongs to an object created from a class.

### STATIC

Belongs to the class itself and is shared at class level.

### MENTAL MODEL

```
dataType variableName = value;
```

# Data types

A data type tells Java what kind of data a variable can store and how that value should be interpreted.



## Eight primitive data types

| Type           | Typical size          | Used for                | Example                    |
|----------------|-----------------------|-------------------------|----------------------------|
| <b>byte</b>    | 8 bits                | small integers          | byte marks = 100;          |
| <b>short</b>   | 16 bits               | larger small integers   | short year = 2026;         |
| <b>int</b>     | 32 bits               | common integers         | int pin = 636123;          |
| <b>long</b>    | 64 bits               | large integers          | long id = 1234567890L;     |
| <b>float</b>   | 32 bits               | decimal values          | float price = 99.5F;       |
| <b>double</b>  | 64 bits               | common decimal values   | double salary = 550000.46; |
| <b>char</b>    | 16 bits               | single UTF-16 code unit | char grade = 'A';          |
| <b>boolean</b> | JVM-dependent storage | true/false state        | boolean success = true;    |

### PRIMITIVE

Stores a primitive value directly according to Java's primitive type rules.

### NON-PRIMITIVE EXAMPLES

**String, arrays, classes and interfaces** are reference types rather than primitive types.

### CORRECTION FROM THE NOTES

**int = 4 bytes = 32 bits** and **long = 8 bytes = 64 bits**. The original note accidentally doubled the bit counts.

# Literals & expressions

## Literals

A **literal** is a fixed value written directly in source code.

```
int age = 20;  
double pi = 3.14;  
char grade = 'A';  
boolean passed = true;  
String msg = "Hello";
```

### SHORTCUT

**Identifier** = name

**Literal** = value written directly

## Expressions

An expression combines variables, literals, operators and sometimes method calls to produce a value.

operand operator operand  
`a + b`

```
int a = 10;  
int b = 10;  
int result = a + b;
```

### RESULT

`a + b` evaluates to 20.

## Operator precedence matters

```
int result = 10 + 10 * 5;
```



### CORRECTION FROM THE NOTES

The expression `10 + 10 * 5` evaluates to **60**, not 100, because multiplication has higher precedence than addition.



# Java operators

An operator is a symbol that performs an operation on one or more operands.

## 1 - ARITHMETIC

`+ - * / %`

Calculations such as addition, subtraction and remainder.

## 2 - ASSIGNMENT

`= += -= *= /= %=`

Assign or update variable values.

## 3 - RELATIONAL

`== != > < >= <=`

Compare two values and produce a boolean.

## 4 - UNARY

`++ -- + - ! ~`

Operate on one operand.

## 5 - LOGICAL

`&& || !`

Combine or reverse boolean conditions.

## 6 - BITWISE

`& | ^ ~ << >>`

Operate on the binary representation of integer values.

## 7 - TERNARY

`condition ? value1 : value2`

Compact conditional expression.

## 8 - INSTANCEOF

Checks whether a reference refers to an object compatible with a given type.

## CLASSIC BEGINNER MISTAKE

`=` means assignment

`==` means equality comparison for primitive values

**Quick Check:** Which operator would you use to find the remainder after division?

# Arithmetic, assignment, relational & logical

## Arithmetic

|   |                |             |
|---|----------------|-------------|
| + | Addition       | 10 + 5 = 15 |
| - | Subtraction    | 10 - 5 = 5  |
| * | Multiplication | 10 * 5 = 50 |
| / | Division       | 10 / 5 = 2  |
| % | Remainder      | 10 % 3 = 1  |

## Assignment

```
int age = 20;
age += 10; // age becomes 30
```

## Relational

```
int a = 10;
int b = 20;
System.out.println(a < b); // true
System.out.println(a == b); // false
```

## Logical

```
int age = 20;
boolean hasId = true;
boolean allowed = age >= 18 && hasId;
```

### MEANING

&& = both true  
 || = at least one true  
 != reverse true/false

## Unary + ternary + instanceof

### UNARY

```
a++;
a--;
!flag;
```

### TERNARY

```
String type = age >
18
    ? "Adult"
    : "Minor";
```

### INSTANCEOF

```
String name =
"Divakar";
boolean isText =
name instanceof
String;
```

# Bitwise operators: the foundation

Bitwise operators work with the binary representation of integer values. Binary uses only the digits **0** and **1**.

## DECIMAL

Our usual number system uses digits 0-9.

19 (decimal)

## BINARY

Binary represents values using powers of two.

0001 0011 (binary) = 19

## Place values

| 128 | 64 | 32 | 16 | 8 | 4 | 2 | 1 |
|-----|----|----|----|---|---|---|---|
| 0   | 0  | 0  | 1  | 0 | 0 | 1 | 1 |

16 + 2 + 1 = 19

## Useful examples

1 = 0001  
 2 = 0010  
 3 = 0011  
 15 = 1111  
 16 = 0001 0000  
 255 = 1111 1111

## INTEGER PRIMITIVE SIZES

**byte 8 bits • short 16 bits • int 32 bits • long 64 bits.** Java also allows bitwise integer operations involving **char**; smaller integer operands are promoted during evaluation.

## WHY LEARN THIS?

Bitwise operations are not used in every beginner program, but they build a strong understanding of how integer values can be manipulated at the bit level.

# AND, OR & XOR

## AND (&)

Result bit is 1 only when both input bits are 1.

| a | b | a & b |
|---|---|-------|
| 0 | 0 | 0     |
| 0 | 1 | 0     |
| 1 | 0 | 0     |
| 1 | 1 | 1     |

## OR (|)

Result bit is 1 when at least one input bit is 1.

| a | b | a   b |
|---|---|-------|
| 0 | 0 | 0     |
| 0 | 1 | 1     |
| 1 | 0 | 1     |
| 1 | 1 | 1     |

## XOR (^)

Result bit is 1 when the two input bits are different.

| a | b | a ^ b |
|---|---|-------|
| 0 | 0 | 0     |
| 0 | 1 | 1     |
| 1 | 0 | 1     |
| 1 | 1 | 0     |

## Example: a = 5, b = 3

5 & 3

```
0101
0011
----
0001
```

Result = 1

5 | 3

```
0101
0011
----
0111
```

Result = 7

5 ^ 3

```
0101
0011
----
0110
```

Result = 6

### CORRECTION FROM THE NOTES

The OR example must use **a | b**. The original example accidentally printed **a & b** while showing the OR result.

# NOT, left shift & right shift

## Bitwise NOT (~)

NOT reverses every bit: **0** -> **1** and **1** -> **0**.

```
int a = 5;
System.out.println(~a); // -6
```

```
5 = 00000000 ... 00000101
~5 = 11111111 ... 11111010
```

### USEFUL IDENTITY

For Java signed integers using two's-complement representation,  **$\sim a = -(a + 1)$** .

## Left shift (<<)

```
int n = 5;
System.out.println(n << 1); // 10
```

```
0101 (5)
1010 (10)
```

Shifting left by one position often corresponds to multiplying by 2 when no relevant overflow occurs.

## Right shift (>>)

```
int n = 10;
System.out.println(n >> 1); // 5
```

```
1010 (10)
0101 (5)
```

For signed integers, **>>** is an arithmetic right shift and preserves the sign bit behavior.

### KEEP IT SIMPLE FOR NOW

At beginner level, focus on the direction of movement and the resulting value. More advanced details such as overflow and unsigned right shift (**>>>**) can be learned later.

# Type casting

Type casting converts a value from one compatible data type to another.

## 1 - WIDENING / IMPLICIT CONVERSION

Java can automatically convert a smaller compatible numeric type to a wider type in many assignments.

```
int age = 20;  
double result = age;  
System.out.println(result);
```

20.0

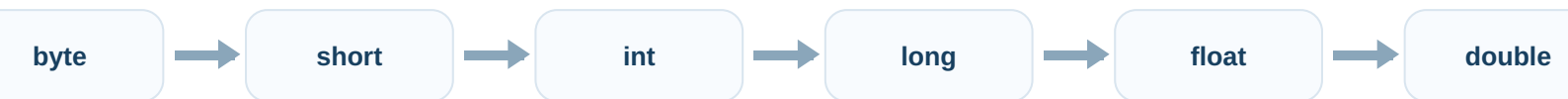
## 2 - NARROWING / EXPLICIT CAST

When converting a wider numeric type to a narrower type, the programmer explicitly writes the target type.

```
double price = 99.99;  
int value = (int) price;  
System.out.println(value);
```

99

## Common widening direction



### IMPORTANT

Narrowing can lose information. Casting **95.75** to **int** gives **95**; Java discards the fractional part rather than rounding it.

**Predict:** What value is stored in `int x = (int) 7.99;`?

# User input with Scanner

**Scanner** is a predefined Java class commonly used by beginners to read input from the keyboard.

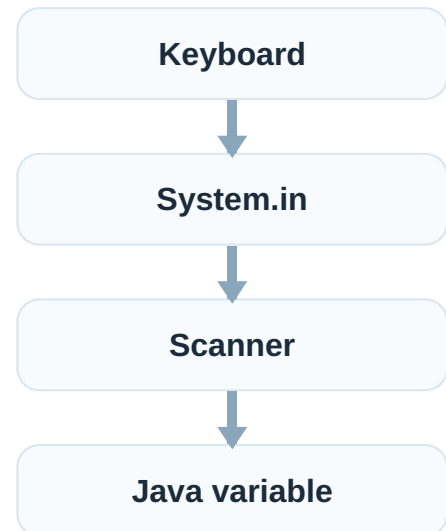
```
import java.util.Scanner;

public class Sample {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int age = sc.nextInt();
        double salary = sc.nextDouble();
        float weight = sc.nextFloat();
        long mobile = sc.nextLong();
        short year = sc.nextShort();
        byte number = sc.nextByte();
        boolean value = sc.nextBoolean();
        char grade = sc.next().charAt(0);
        String name = sc.next();

        sc.close();
    }
}
```

## Input flow



|               |                |
|---------------|----------------|
| nextInt()     | int            |
| nextDouble()  | double         |
| nextFloat()   | float          |
| nextLong()    | long           |
| nextShort()   | short          |
| nextByte()    | byte           |
| nextBoolean() | boolean        |
| next()        | one token/word |

### CORRECTIONS FROM THE NOTES

Use primitive types **byte** and **boolean** here, not wrapper types unless intentionally needed. To read one character, use **sc.next().charAt(0)**.

# Scanner: next() vs nextLine()

## NEXT()

Reads the next token, usually stopping at whitespace.

```
String firstName = sc.next();
```

Input: **Arun Kumar**  
Stored value: **Arun**

## NEXTLINE()

Reads the remainder of the current line, including spaces until the newline.

```
String fullName = sc.nextLine();
```

Input: **Arun Kumar**  
Stored value: **Arun Kumar**

## The common newline problem

```
Scanner sc = new Scanner(System.in);

System.out.print("Enter age: ");
int age = sc.nextInt();

sc.nextLine(); // consume the leftover newline

System.out.print("Enter full name: ");
String fullName = sc.nextLine();
```

### WHY THIS HAPPENS

**nextInt()** reads the integer but leaves the line-ending character pending. A following **nextLine()** can consume that line ending immediately unless you handle it.

**Quick Check:** If a student's name can contain spaces, which Scanner method is more suitable: **next()** or **nextLine()**?



# Output methods

Java's standard output methods display information in the console.

## PRINT()

Prints without automatically moving to the next line.

```
System.out.print("Hello");  
System.out.print("Java");
```

```
Hello Java
```

## PRINTLN()

Prints and then moves to the next line.

```
System.out.println("Hello");  
System.out.println("Java");
```

```
Hello Java
```

## PRINTF()

Prints formatted output using placeholders.

```
String name = "Divakar";  
System.out.printf("Name: %s\n", name);
```

```
Name: Divakar
```

## Common format specifiers

| Specifier | Typical use             | Example  |
|-----------|-------------------------|----------|
| %s        | String/text             | "Arun"   |
| %d        | decimal integer         | 95       |
| %f        | floating-point          | 8.250000 |
| %n        | platform line separator | new line |

## FORMATTING DECIMALS

```
double cgpa = 8.2567;  
System.out.printf("CGPA: %.2f\n", cgpa);
```

```
CGPA: 8.26
```

# Code walkthrough: Solution + Module2

Your final example combines classes, methods, object creation, operators and output. You do not need to master OOP yet - just observe how the fundamentals appear together.

```
class Solution {
    public int sum(int num1, int num2) {
        return num1 + num2;
    }

    public int sub(int num1, int num2) {
        return num1 - num2;
    }

    public int mul(int num1, int num2) {
        return num1 * num2;
    }

    public int div(int num1, int num2) {
        return num1 / num2;
    }
}

public class Module2 {
    public static void main(String[] args) {
        Solution sol = new Solution();
        System.out.println(sol.sum(12, 5));
    }
}
```

## WHAT HAPPENS?

- 1 **new Solution()** creates a Solution object.
- 2 **sol.sum(12, 5)** calls the sum method.
- 3 **12 + 5** evaluates to 17.
- 4 **println** displays 17.

## OUTPUT

17

The **div** method uses integer division because both parameters and the return type are **int**.

# Quick recap + practice

## MEMORY MAP

Token = smallest recognized unit

Keyword = reserved word

Identifier = programmer-created name

Variable = named storage

Data type = kind of value

Literal = direct value

Operator = action symbol

Expression = produces a value

Casting = type conversion

Scanner = input

## Predict the output

```
int a = 10;
int b = 3;
System.out.println(a / b);
System.out.println(a % b);
```

### THINK

Both operands are integers. What does integer division produce?

### READY TO MOVE FORWARD

When you can explain these concepts and write the final practice program without copying, you have the foundation needed for control statements, loops, methods and arrays.

## Practice

1. Break **double price = 99.5;** into tokens.

2. Write one valid class name, method name and constant name using Java conventions.

3. Choose suitable primitive types for age, CGPA, section letter and login state.

4. Predict **5 & 3**, **5 | 3** and **5 ^ 3**.

5. What is stored by **int x = (int) 12.99;**?

6. Explain why **nextLine()** may appear to skip input after **nextInt()**.

7. Write a program that reads a student's name and three marks, calculates the total and prints formatted output.